TDD (Test Driven Development)

Clases: 25/09/2026 y 07/10/2026

1. Dónde se ubica TDD

TDD trabaja sobre el nivel más bajo de prueba: el testing unitario. Los niveles de prueba son unitarias, de integración, de sistema y de aceptación.

En el testing unitario:

- Se prueba cada componente después de construirlo.
- Cada componente se prueba de forma individual e independiente.
- Se hace con acceso al código y con apoyo del entorno de desarrollo (framework o herramientas).
- Los errores se suelen corregir apenas se encuentran, sin registro formal de lo que paso.

TDD junta las pruebas y las hace antes del código, no después.

TDD viene de Kent Beck y de XP (Extreme Programming).

2. Qué es TDD

TDD técnica avanzada que combina dos prácticas:

- **Test First Development:** escribir la prueba primero.
- **Refactoring:** mejorar el código sin cambiar su comportamiento.

Para escribir las pruebas generalmente se usan pruebas unitarias.

TDD no es testing, es desarrollo. Es una actividad de especificación y diseño: se dice QUE se tiene que hacer antes del COMO

Se pasa de un enfoque reactivo a uno proactivo.

3. Desarrollo tradicional vs. TDD

Desarrollo tradicional: Diseñar → Desarrollar → Probar.

- Los desarrolladores diseñan las pruebas unitarias después de escribir el código.
- Reactivo.
- negocio, requerimientos, análisis, diseño, implementación y, al final, prueba.

TDD: Diseñar → Probar → Desarrollar.

- Los desarrolladores diseñan las pruebas unitarias antes de escribir el código.
- El aseguramiento y control de calidad es proactivo.

El problema de probar al final. Tres consecuencias

- **Acoplamiento incidental:** el código queda acoplado y es difícil de testear.
- **Deuda técnica y miedo al cambio:** Nadie se anima a modificar el código.
- **Costo asintótico del error:** cuanto más tarde se detecta un defecto, más caro es corregirlo.

4. Las 3 leyes de TDD

- No escribas una línea de código de producción sin antes haber escrito un test unitario que falle.
- No escribas más código de prueba que el necesario para provocar un fallo. Si el código ni siquiera compila.
- No escribirás más código de producción que el mínimo indispensable para que el test pase.

5. El ciclo Red-Green-Refactor

Es el corazón de TDD.

- **RED:** escribo un test que falla. Es de manera declarativa: declaro la intención ("quiero que al pasarle ESTOS datos, el sistema me devuelva ESTE resultado"). La clase todavía no está implementada.

- **GREEN:** Enfoque imperativo (el cómo) y escribo el código mínimo para que la prueba pase, aunque sea "trucho".
- **REFACTOR:** modifico la estructura interna del software sin alterar su comportamiento externo. Las pruebas tienen que seguir pasando.

Después, el ciclo comienza de nuevo con otro test.

Pasos a paso del ciclo

1. Escribir un test.
2. Hacer que el test compile.
3. Hacer que el test falle (Red).
4. Hacer un cambio para que el test pase (Green).
5. Correr el test para comprobar que pasa.
6. Quitar duplicaciones (Refactor).

Estructura de un test

- **Precondiciones:** datos y estado necesarios antes de ejecutar.
- **Pasos:** la acción que se ejecuta.
- **Resultado:** los assert que verifican lo esperado.

Ejemplo: Calculadora
RED: el test pide sumar 1 + 2 y espera 3. El método sumar todavía no está implementado, así que falla.

```
def test_sumar_dos_numeros_EXITO():
    // Precondiciones
    calculadoraDePrueba = new Calculadora()
    // Pasos
    resultado = calculadoraDePrueba.sumar(a = 1, b = 2)
    // Resultado
    assert(resultado == 3)

Class Calculadora:
    def sumar(a, b):
        return "No implementado"
```

GREEN: hago lo mínimo para que pase, devolviendo un valor fijo (patrón Fake It).

```
Class Calculadora:
    def sumar(a, b):
        return 3
```

REFACTOR: mejoro el código sin cambiar el comportamiento; por ejemplo, agrego los tipos de los parámetros.

```
Class Calculadora:
    def sumar(a: Number, b: Number) -> Number:
        return 3
```

NUEVO CICLO: agrego otro test con datos distintos (3 + 2 = 5). El "return 3" ya no sirve, así que me veo forzado a generalizar (patrón Triangulación).

```
    resultado = calculadoraDePrueba.sumar(a = 3, b = 2)
    assert(resultado == 5)

Class Calculadora:
    def sumar(a: Number, b: Number) -> Number:
        return a + b
```

6. Patrones de TDD
Son estrategias o trucos que usa el programador para pasar de una fase a otra dentro del ciclo.

<u>Patrones Red Bar</u>

- **One Step Test:** un solo test que verifica una pieza mínima de funcionalidad.
- **Starter Test:** primer test, sirve para explorar la API o el comportamiento deseado.
- **Explanation Test:** comentarios o preguntas para aclarar qué se espera que haga el código.
- **Learning Test:** permite conocer APIs externas cuyo comportamiento no conocemos.
- **Another Test:** agrega un test para cubrir un caso adicional o una variante.
- **Regression Test:** reproduce un bug descubierto, para evitar que vuelva a aparecer.

Patrones Green Bar

- **Fake It ('til you make it):** devolver un valor fijo o una solución "falsa" solo para que el test pase.
- **Triangulación:** agregar un segundo test con otros datos para forzar una implementación más general.
- **Obvious Implementation:** si la solución es trivial y obvia, se implementa directamente, sin Fake It.
- **One to Many:** para resolver tests sobre colecciones de elementos.

Patrones de diseño de tests

- **Test:** nombrar las pruebas con la palabra test y la clase a probar.
- **Isolated Test:** los tests son totalmente independientes entre sí.
- **Test List:** primero se escribe la lista de tests que se van a realizar.
- **Test First:** se escribe primero la prueba.
- **Assert Test:** primero se escribe la confirmación (assert) y después el resto de la prueba.
- **Test Data:** datos de prueba fáciles de leer y seguir.
- **Evident Data:** incluir los resultados esperados y reales en el test, con una relación evidente entre los datos.

Otros patrones
Patrones de testing, patrones de diseño y xUnit Patterns (assertion, fixture, external fixture, test method, exception test, all tests).

7. Refactoring
Es una forma disciplinada de introducir cambios reduciendo la posibilidad de introducir defectos. También es una transformación del software que preserva la estructura externa y mejora la estructura interna.

Son cambios que:

- No cambian el comportamiento observable (todas las pruebas siguen pasando).
- Eliminan la duplicación o la complejidad innecesaria.
- Mejoran la calidad del software.
- Hacen el código más simple, más fácil de entender y más fácil de cambiar (más flexible).

Refactorizar sirve para:

- Para evitar el "deterioro del diseño".
- Para limpiar el desorden y simplificar el código.
- Para aumentar la legibilidad y la comprensibilidad.
- Para encontrar errores y reducir el tiempo de depuración.
- Para incorporar lo que aprendemos sobre la aplicación (rehacer es fundamental en todo proceso creativo).

Cómo refactorizar?

1. Asegurarse de que las pruebas pasen (TDD).
2. Encontrar código que "huela mal".
3. Determinar cómo simplificarlo.
4. Hacer las simplificaciones.
5. Ejecutar las pruebas para asegurarse de que siguen pasando.

8. Beneficios de TDD

- Código robusto, más predecible y limpio; permite saber cuándo el código está terminado.
- Código más tolerante al cambio y más fácil de entender, porque las pruebas ya están escritas.
- Código más seguro.
- Menos duplicación: si el test pasa, el código ya existe.
- Código más barato de mantener, al ser más entendible.
- Con la práctica acelera el desarrollo y facilita el despliegue continuo.
- Asegura cobertura de prueba, lo que lleva a código de calidad.
- Obliga a pensar cómo se va a usar el componente, y por lo tanto el diseño de las API.
- Promueve componentes con alta cohesión y bajo acoplamiento.

9. Caso práctico: Taxi Mobile (clase 07/10)

Se aplicó el ciclo a la User Story "Ver mapa de taxis": como Administrador de la Central quiero ver la ubicación de todos los taxis y si tienen viajes en curso, para saber la disponibilidad actual.

RED: test "mostrar mapa de taxis libres en barrio (éxito)".

- Precondiciones: un usuario con rol Administrador de la Central logueado, una ciudad de prueba (Córdoba Capital) y un taxi cargado (chapa ABC123, estado Libre, barrio General Paz).
- Pasos: mostrar_mapa con filtros por barrio y estado (barrio = General Paz, estado = Libre).
- Resultado: asserts de que el barrio existe y de que todos los taxis mostrados están en General Paz, están Libres y tienen el ícono verde.
- La función mostrar_mapa existe pero está vacía (pass), así que el test falla.

GREEN: mostrar_mapa devuelve directamente un Mapa con ese taxi fijo (Fake It). El test pasa.

REFACTOR: se tipan los parámetros (Filtros, Barrio, Estado, Mapa) y se reemplazan los datos sueltos por objetos (Estado Libre con ícono verde, Barrio General Paz, Taxi ABC123). El comportamiento no cambia y el test sigue pasando.

Después se vuelve a iniciar el ciclo, agregando uno por uno el resto de los casos de prueba de la US (las pruebas de usuario), y se refactoriza en cada vuelta. En el ejemplo final, mostrar_mapa terminó contemplando:

- Taxis desconectados (sin ubicación).
- Taxis por estado y por barrio.
- Búsqueda por chapa.
- Taxis por barrio con datos del viaje (pasajero, teléfono, fecha y hora).

Las pruebas de usuario de la User Story son la fuente de los tests que se van escribiendo en cada ciclo.